



Python Modeling Cheat Sheet

Organized by Model | Explained for Your Datasets | Long-Term Utility



Step 1: Universal Setup – Preprocessing for Any Dataset



Import Libraries

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import (
    accuracy_score, confusion_matrix, classification_report,
    mean_squared_error, r2_score
)
import joblib
```



Load & Inspect Dataset

```
df = pd.read_csv('your_data.csv')
print(df.head())
print(df.info())
print(df.describe())
```



Handle Missing Values

```
df.fillna(method='ffill', inplace=True)
# OR: df.dropna(inplace=True)
```



Encode Categorical Features

```
categorical_cols = df.select_dtypes(include=['object', 'category']).columns
le = LabelEncoder()
for col in categorical_cols:
    df[col] = le.fit_transform(df[col])
```



Define Features & Target

```
X = df.drop('target_column', axis=1)
y = df['target_column']
```



Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```



Feature Scaling (where applicable)

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```



1. Logistic Regression

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()
model.fit(X_train_scaled, y_train)

y_pred = model.predict(X_test_scaled)
```

Evaluation:

```
print(accuracy_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

Tuning:

```
param_grid = {'C': [0.1, 1, 10]}
grid = GridSearchCV(LogisticRegression(), param_grid, cv=5)
grid.fit(X_train_scaled, y_train)
```

Save:

```
joblib.dump(model, 'logreg_model.pkl')
```



2. Decision Tree

Classification

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(max_depth=5)
model.fit(X_train, y_train)
```

Regression

```
from sklearn.tree import DecisionTreeRegressor

model = DecisionTreeRegressor(max_depth=5)
model.fit(X_train, y_train)
```

Evaluation & Tuning: same structure as Logistic Regression



3. Random Forest

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(n_estimators=100)
model.fit(X_train, y_train)
```

Tuning:

```
param_grid = {'n_estimators': [100, 200], 'max_depth': [5, 10, None]}
grid = GridSearchCV(RandomForestClassifier(), param_grid, cv=5)
grid.fit(X_train, y_train)
```



4. Support Vector Machine (SVM / SVR)

Classification

```
from sklearn.svm import SVC

model = SVC(kernel='rbf')
model.fit(X_train_scaled, y_train)
```

Regression

```
from sklearn.svm import SVR

model = SVR(kernel='rbf')
model.fit(X_train_scaled, y_train)
```



5. Linear Regression

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train_scaled, y_train)

y_pred = model.predict(X_test_scaled)
```

Eval:

```
print(mean_squared_error(y_test, y_pred))
print(r2_score(y_test, y_pred))
```

6. Neural Network (Keras)

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

model = Sequential([
    Dense(64, activation='relu', input_shape=(X_train_scaled.shape[1],)),
    Dense(64, activation='relu'),
    Dense(1, activation='sigmoid') # Binary classification
])

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
model.fit(X_train_scaled, y_train, epochs=10, batch_size=32)
```

7. KMeans Clustering (Unsupervised)

Fit Model

```
from sklearn.cluster import KMeans

kmeans = KMeans(n_clusters=3, random_state=42)
kmeans.fit(X_train_scaled)

labels = kmeans.labels_
centroids = kmeans.cluster_centers_
```

Cluster Evaluation Metrics

Silhouette Score

```
from sklearn.metrics import silhouette_score

score = silhouette_score(X_train_scaled, labels)
print("Silhouette Score:", score)
```

Calinski-Harabasz Score

```
from sklearn.metrics import calinski_harabasz_score

ch_score = calinski_harabasz_score(X_train_scaled, labels)
print("Calinski-Harabasz Score:", ch_score)
```

Davies-Bouldin Score

```
from sklearn.metrics import davies_bouldin_score

db_score = davies_bouldin_score(X_train_scaled, labels)
print("Davies-Bouldin Score:", db_score)
```



Cluster Visualization

```
import matplotlib.pyplot as plt

plt.scatter(X_train_scaled[:, 0], X_train_scaled[:, 1], c=labels)
plt.title("KMeans Clusters")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```



Optional: PCA for 2D Visualization

```
from sklearn.decomposition import PCA

X_pca = PCA(n_components=2).fit_transform(X_train_scaled)

plt.scatter(X_pca[:, 0], X_pca[:, 1], c=labels)
plt.title("KMeans Clusters in 2D")
plt.show()
```